

“The Static Method Thing” at <https://tomayko.com/blog/2004/the-static-method-thing>.

Object-Oriented Buzzwords

Explanations of OOP often begin with a lot of jargon, such as inheritance, encapsulation, and polymorphism. The importance of knowing these terms is overrated, but you should have at least a basic understanding of them. I already covered inheritance, so I’ll describe the other concepts here.

Encapsulation

The word *encapsulation* has two common but related definitions. The first definition is that encapsulation is the bundling of related data and code into a single unit. To encapsulate means to *box up*. This is essentially what classes do: they combine related attributes and methods. For example, our `WizCoin` class encapsulates three integers for knuts, sickles, and galleons into a single `WizCoin` object.

The second definition is that encapsulation is an *information hiding* technique that lets objects hide complex implementation details about how the object works. You saw this in “Private Attributes and Private Methods” on page 282, where `BankAccount` objects present `deposit()` and `withdraw()` methods to hide the details of how their `_balance` attributes are handled. Functions serve a similar *black box* purpose: how the `math.sqrt()` function calculates the square root of a number is hidden. All you need to know is that the function returns the square root of the number you passed it.

Polymorphism

Polymorphism allows objects of one type to be treated as objects of another type. For example, the `len()` function returns the length of the argument passed to it. You can pass a string to `len()` to see how many characters it has, but you can also pass a list or dictionary to `len()` to see how many items or key-value pairs it has, respectively. This form of polymorphism is called *generic functions* or *parametric polymorphism*, because it can handle objects of many different types.